

Reviewer Pack — Free Entropy Lower Bound on Disjointness Communication Compl...

Entry cd1afb38ca62 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 15:00:44 UTC

Free Entropy Lower Bound on Disjointness Communication Complexity

Entry ID: cd1afb38ca62 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 15:00:44 UTC

1. Conjecture

Field A (mathematical branch): FREE_PROBABILITY **Field B** (complexity object): COMMUNICATION_COMPLEXITY

Statement:

For any $n \geq 1$, let M_n be the communication matrix of the DISJOINTNESS function on n -bit inputs. Define $\tau(M_n)$ as the free entropy of the non-commutative random variables derived from M_n 's spectral decomposition. Then $\tau(M_n) \geq \Omega(n)$ and $\tau(\text{DISJ}_n) = \Omega(n)$ for all n .

Rationale (proposer's reasoning):

Free entropy quantifies the 'complexity' of non-commutative distributions, capturing dependencies in high-dimensional matrices. By linking M_n 's spectral structure to free probability invariants, we derive a lower bound on communication complexity via the non-commutative nature of DISJOINTNESS's matrix representation.

Taxonomy category: AVG_TO_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2115ca22be56d983

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            raise ValueError("Matrix is singular")
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def spectral_measure(A):
    n = len(A)
    eigenvalues = []
    for _ in range(10): # Power iteration method
        v = [random.random() for _ in range(n)]
        v /= math.sqrt(sum(x**2 for x in v))
        for _ in range(100):
            v = matrix_multiplication(A, v)
            v /= math.sqrt(sum(x**2 for x in v))
        eigenvalues.append(v[0])
    return eigenvalues
```

```

def free_entropy(M_n):
    A = gaussian_elimination(M_n)
    eigenvalues = spectral_measure(A)
    tau_M_n = -sum(eigenvalue * math.log(abs(eigenvalue)) for eigenvalue in eigenvalues if eigenvalue != 0)
    return tau_M_n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    M_n = [[random.random() - 0.5 for _ in range(n)] for _ in range(n)]
    tau_M_n = free_entropy(M_n)
    return {
        "metric_name": "free_entropy",
        "metric_value": tau_M_n,
        "instances_tested": n,
        "conjecture_holds": tau_M_n >= n,
        "counterexample": "" if tau_M_n >= n else f"tau_M_{n} = {tau_M_n}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\"}, \"metric_value\": {result['metric_value']}}")
        results.append(result)

    mean = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3435fa78.py", line 75, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3435fa78.py", line 62, in run_trial
    tau_M_n = free_entropy(M_n)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3435fa78.py", line 53, in free_entropy

```

```

A = gaussian_elimination(M_n)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3435fa78.py", line 26, in gaussian_elim
factor = Fraction(A[j][i], A[i][i])
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/fractions.py", line 277, in __new__
raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during fraction conversion, preventing data collection. The error suggests a type mismatch in the Gaussian elimination implementation. | next: Debug the Fraction type conversion in gaussian_elimination() by ensuring all inputs are Rational instances before division

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	42348
2	propose	ollama_remote	qwen3:8b	0	0	73688
3	propose	ollama_remote	qwen3:8b	0	0	52507
4	propose	ollama_remote	qwen3:8b	0	0	53863
5	preregistration	ollama_remote	qwen3:8b	0	0	24009
6	novelty	ollama_remote	qwen3:8b	0	0	20727
7	novelty	ollama_remote	qwen3:8b	0	0	15664
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16039
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10165
10	judge	ollama_remote	qwen3:8b	0	0	18388

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 327398 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/cd1afb38ca62.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cd1afb38ca62.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cd1afb38ca62.tar.gz` (if generated)